

Clasificación de imágenes de Fashion-MNIST usando PyTorch

Brunello Florencia Luciana,^{*} Lotumolo Lucía,^{**} and Silva Walter Andre^{***}

Facultad de Matemática, Astronomía, Física y Computación,

Universidad Nacional de Córdoba, Ciudad Universitaria, 5000 Córdoba, Argentina

(Dated: 11 de noviembre de 2025)

En este trabajo se implementó y analizó una red neuronal feedforward destinada a la clasificación del conjunto de datos Fashion-MNIST, una versión moderna del clásico dataset MNIST, enfocada en el reconocimiento de prendas de vestir. El modelo fue desarrollado utilizando la biblioteca PyTorch, y se realizaron múltiples experimentos con el objetivo de estudiar cómo la variación de distintos hiperparámetros de entrenamiento, tales como la tasa de aprendizaje, el optimizador, el porcentaje de dropout, el tamaño del batch, la cantidad de neuronas por capa y el número de épocas, influye en el rendimiento y la capacidad de generalización del modelo. Se observó que combinaciones moderadas de tasa de aprendizaje, dropout y tamaño de batch, junto con el uso del optimizador Adam, conducen a un modelo con alto rendimiento y convergencia eficiente en la tarea de clasificación de imágenes.

I. INTRODUCCIÓN

En la actualidad, el reconocimiento y la clasificación automática de imágenes constituyen áreas de gran interés dentro del campo de la inteligencia artificial. En este contexto, las redes neuronales artificiales feedforward multicapa (Multilayer Perceptron, MLP) representan una de las arquitecturas fundamentales del aprendizaje supervisado. Este tipo de red está conformado por un conjunto de capas de neuronas artificiales conectadas de manera unidireccional, desde la capa de entrada hasta la capa de salida, sin conexiones de retroalimentación. Cada neurona combina linealmente sus entradas y aplica una función de activación no lineal, lo que permite a la red capturar y modelar relaciones complejas entre los datos [1].

El entrenamiento de estas redes se realiza mediante el algoritmo de retropropagación del error, que ajusta los pesos sinápticos para minimizar una función de pérdida y mejorar la capacidad predictiva del modelo [2]. Gracias a esta estructura y proceso de aprendizaje, las redes feedforward multicapa pueden emplearse eficazmente en tareas de clasificación supervisada, donde el modelo aprende a identificar patrones en los datos de entrada y asignar cada muestra a una categoría específica.

El objetivo de este informe es implementar y analizar una red neuronal feedforward multicapa aplicada al conjunto de datos Fashion-MNIST, con el propósito de evaluar su desempeño en la clasificación de imágenes y estudiar la influencia de distintos hiperparámetros de entrenamiento.

II. TEORÍA

El modelo utilizado corresponde a una red neuronal feedforward multicapa, compuesta por una capa de entrada de tamaño $n_I = 28 \times 28 = 784$, dos capas ocultas de tamaños n_1 y n_2 , y una capa de salida de $n_O = 10$ neuronas, correspondientes a las diez categorías de prendas presentes en el conjunto de datos Fashion-MNIST, donde las capas se encuentran totalmente conectadas entre sí.

En las capas ocultas se utilizó la función de activación

ReLU, que introduce no linealidad al modelo y facilita la propagación del gradiente durante el entrenamiento. Para mitigar el sobreajuste, se incorporó una capa de dropout con probabilidad p , que desactiva aleatoriamente un porcentaje de neuronas durante el entrenamiento. Aunque el dropout puede empeorar temporalmente las predicciones durante el entrenamiento, permite reducir la dependencia entre las neuronas, mejorando la capacidad de generalización del modelo en datos no vistos y previniendo el sobreajuste.

El entrenamiento de la red se realizó mediante el algoritmo de retropropagación del error, utilizando la función de pérdida *Cross Entropy Loss*, adecuada para problemas de clasificación multiclase. Este criterio mide la diferencia entre la distribución de probabilidades predicha por la red y la verdadera distribución de etiquetas, y su minimización permite ajustar los pesos sinápticos para mejorar la precisión de la clasificación.

El proceso de entrenamiento se estructuró en épocas. En cada época, la red recorre todo el conjunto de entrenamiento dividido en lotes de tamaño fijo. Este enfoque permite procesar los datos de manera eficiente, aprovechando el paralelismo de la GPU y reduciendo la varianza del gradiente.

El conjunto de datos con el que se trabajó es Fashion-MNIST, que consiste en 70.000 imágenes de prendas en escala de grises de 28×28 píxeles, divididas en 60.000 muestras para entrenamiento y 10.000 para validación. Cada imagen está asociada a una etiqueta representada por un número entero entre 0 y 9 que identifica la categoría (por ejemplo: remera, pantalón, zapatilla, etc.). El conjunto de entrenamiento se utiliza para ajustar los pesos sinápticos de las neuronas mientras que el conjunto de validación no interviene en el ajuste de los parámetros. Su función es medir el desempeño del modelo sobre datos que no han sido utilizados en el entrenamiento. Si la pérdida sobre el conjunto de validación comienza a aumentar mientras que la pérdida en el de entrenamiento sigue disminuyendo, significa que el modelo está memorizando los ejemplos del conjunto de entrenamien-

to y aprendiendo el ruido en lugar de aprender patrones generales, es decir, no puede generalizar bien. Los datos fueron aleatorizados (*shuffle=True*) antes de generar los *DataLoaders*, asegurando que ambos conjuntos sean representativos de todas las clases y que el orden de las muestras no introduzca sesgos.

III. RESULTADOS

Para comenzar con la experimentación, se decidió tomar un modelo base con las siguientes configuraciones: *learning rate* = 1×10^{-3} , *optimizador* = *Adam*, *dropout* = 0,2, $n_1 = 128$, $n_2 = 64$, *epocas* = 50, *batch* = 100. Cada uno de estos hiperparámetros fue modificado de manera independiente con respecto a los demás, con el objetivo de analizar su comportamiento de forma individual. La Figura 1 presenta los resultados obtenidos con esta configuración inicial, mientras que las Figuras 2, 3, 4, 5 y 6 muestran los resultados obtenidos al variar cada uno de los hiperparámetros mencionados.

IV. CONCLUSIONES

A continuación, analizamos el comportamiento de cada uno de los hiperparámetros a partir de los resultados obtenidos en los gráficos durante los experimentos.

Tasa de aprendizaje: Al ver la Figura 2 notamos que con un learning rate bajo (1×10^{-4}) el entrenamiento resulta estable y sin oscilaciones, aunque la convergencia es lenta y requiere muchas épocas para alcanzar un buen rendimiento. En cambio, con un learning rate alto (1×10^{-2}) se observan grandes fluctuaciones en la pérdida y la precisión a lo largo del entrenamiento, e incluso en varias épocas la pérdida en el conjunto de validación aumenta, lo que indica que el modelo oscila alrededor de los mínimos en lugar de converger. Por último, el learning rate intermedio (1×10^{-3}) muestra el mejor equilibrio entre estabilidad y velocidad de aprendizaje, logrando buenos resultados en menor tiempo, aunque si se prolonga el entrenamiento podría comenzar a sobreajustar.

Optimizador: Al comparar los resultados de la Figura 3 se observa una diferencia clara tanto en la velocidad de convergencia como en el rendimiento final. Con SGD, el modelo mejora de forma progresiva y estable, aunque más lentamente, mientras que con Adam el entrenamiento es considerablemente más rápido y logra un mejor rendimiento general. Con SGD, una tasa pequeña (0.001) produce un entrenamiento estable pero lento, mientras que con 0.01 la convergencia mejora. Al usar una tasa mayor de 0.05, el modelo alcanza un rendimiento comparable al de Adam. Por su parte, Adam ajusta dinámicamente la tasa de aprendizaje, acelerando o estabilizando el proceso según el comportamiento del error, lo que le permite converger más rápido y mantener un buen rendimiento general.

Dropout: La Figura 4 muestra que con $p = 0,0$, a

partir de la época 25 el modelo tiende a sobreajustar, alcanzando alta precisión en el conjunto de entrenamiento pero menor desempeño en el de validación. Con $p = 0,5$ se reduce el sobreajuste y mejora la capacidad de generalización, aunque el entrenamiento se vuelve más lento. El valor intermedio $p = 0,2$ resultó el más equilibrado entre rendimiento y estabilidad.

Número de neuronas por capa: En la Figura 5(a) se puede apreciar la escasa variación en los conjuntos de validación, aunque sí se observa una diferencia notable en el conjunto de entrenamiento. Al analizar la precisión (Figura 5(b)), se nota que, a medida que aumenta el número de neuronas por capa, las curvas correspondientes al conjunto de entrenamiento y al de validación tienden a separarse. En el caso 3, se evidencia que el modelo comienza a memorizar el conjunto de entrenamiento, sin mejorar la capacidad de generalización en el conjunto de validación.

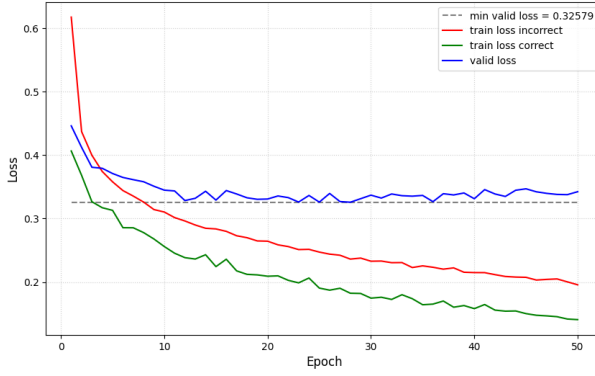
Número de épocas: Como se puede apreciar en los distintos gráficos, por ejemplo, en la Figura 5(a), entrenar durante más de 30 épocas no produce mejoras significativas en la predicción. Además, este exceso puede resultar contraproducente, como se observa en la Figura 4(a), donde el error acumulado aumenta progresivamente después de superar las 20 épocas de entrenamiento.

Tamaño del batch: La Figura 6 muestra el efecto del tamaño del *batch*. Con *batch-size* = 20, el aprendizaje presenta mayor variabilidad entre épocas pero mejor generalización en validación. En cambio, con *batch-size* = 2000, el proceso es más estable aunque el modelo tiende a sobreajustar los datos. El valor intermedio *batch-size* = 100 ofrece un equilibrio adecuado entre estabilidad y capacidad de generalización.

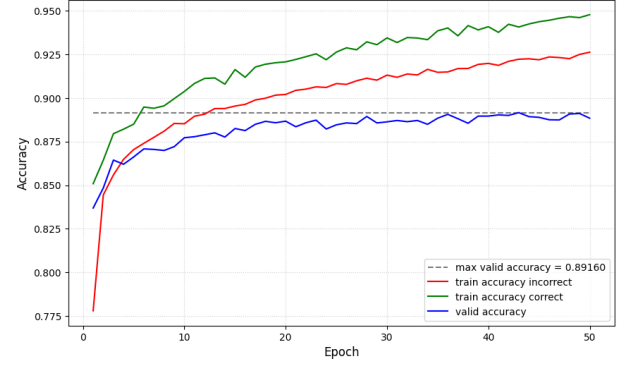
Tras realizar varias pruebas modificando distintos parámetros de manera simultánea, podemos concluir que los hiperparámetros establecidos como base para el modelo de la red neuronal mantienen un comportamiento estable siempre que las variaciones sean pequeñas y no se lleven a valores extremos. Sobre el conjunto de validación, se obtuvo una pérdida de 0.32 y una precisión del 89 % aproximadamente (véase la Figura 1).

V. REFERENCIAS

-
- * florenciabrunello@unc.edu.ar
 - ** lucia.lotumolo@mi.unc.edu.ar
 - *** walter.andre.silva@mi.unc.edu.ar
 - [1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning* (MIT Press, 2016).
 - [2] S. Haykin, *Neural Networks and Learning Machines (3rd ed.)* (Pearson Education, 2009).

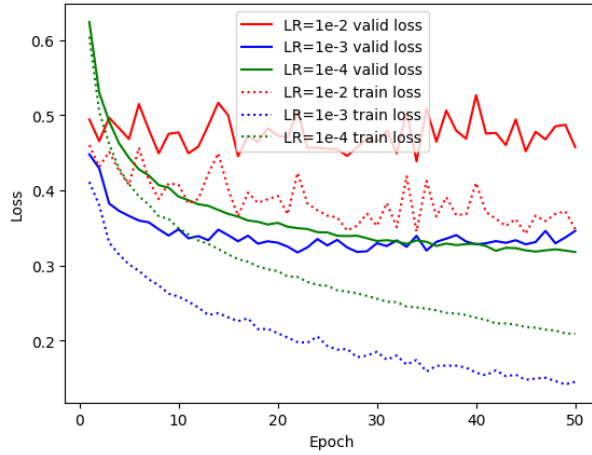


(a) Pérdida para los distintos valores de learning rate

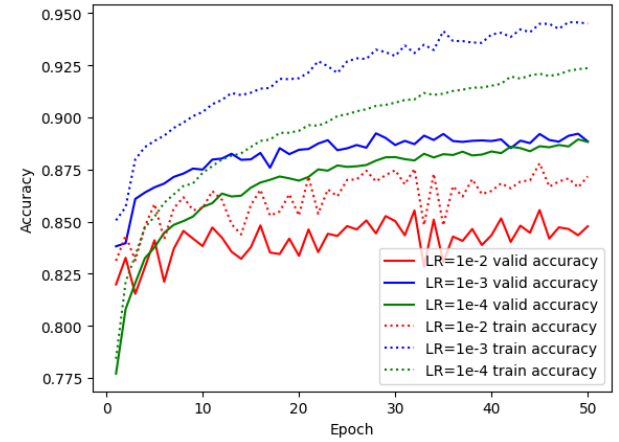


(b) Precisión para los distintos valores de learning rate

Figura 1. **Modelo base.** Learning rate = 1×10^{-3} , optimizador = Adam, dropout = 0,2, n1 = 128, n2 = 64, épocas = 50, batch = 100

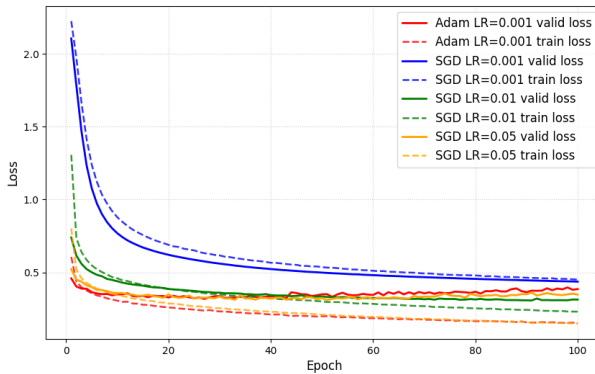


(a) Pérdida para los distintos valores de learning rate

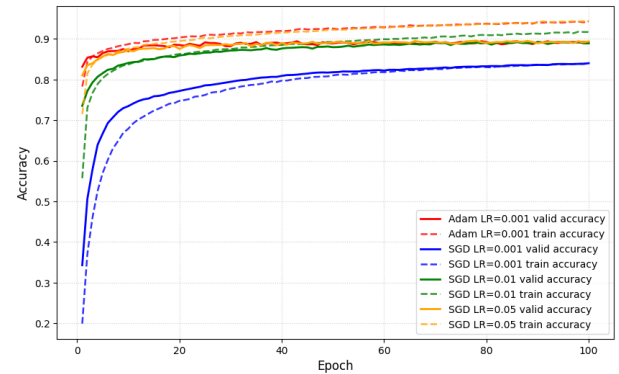


(b) Precisión para los distintos valores de learning rate

Figura 2. **Valores de learning rate.** Caso 1: $lr = 1 \times 10^{-2}$; Caso 2: $lr = 1 \times 10^{-3}$; Caso 3: $lr = 1 \times 10^{-4}$;

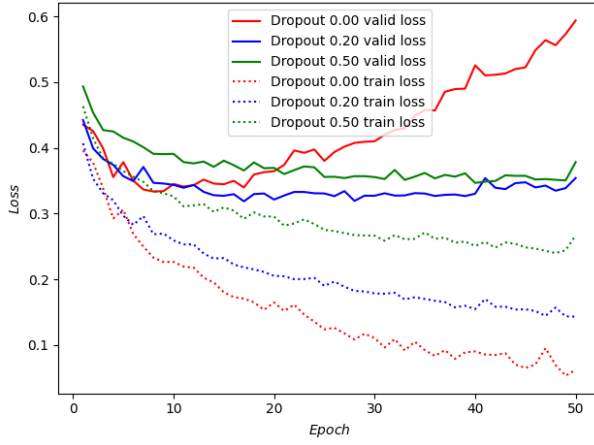


(a) Pérdida para los distintos optimizadores

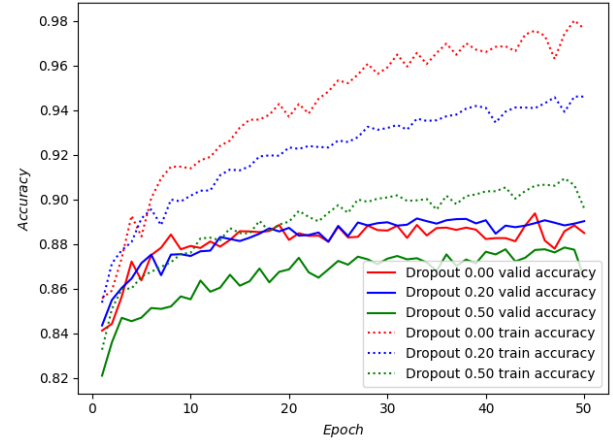


(b) Precisión para los distintos optimizadores

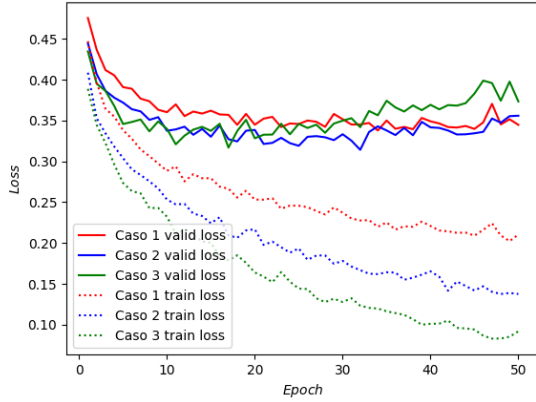
Figura 3. **Optimizadores.** Caso 1: Adam con $lr = 0,001$; Caso 2: SGD con $lr = 0,001$; Caso 3: SGD con $lr = 0,01$; Caso 4: SGD con $lr = 0,05$.



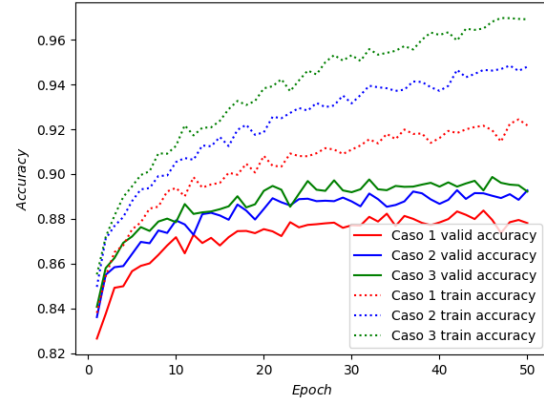
(a) Pérdida para los distintos valores de dropout



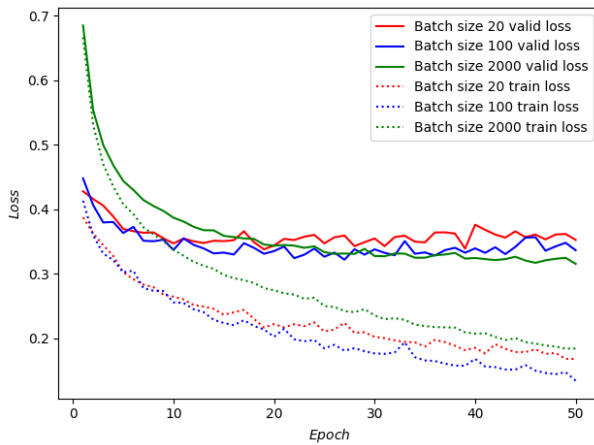
(b) Precisión para los distintos valores de dropout

Figura 4. **Valores de dropout.** Caso 1: $dropout = 0,00$; Caso 2: $dropout = 0,20$; Caso 3: $dropout = 0,50$;

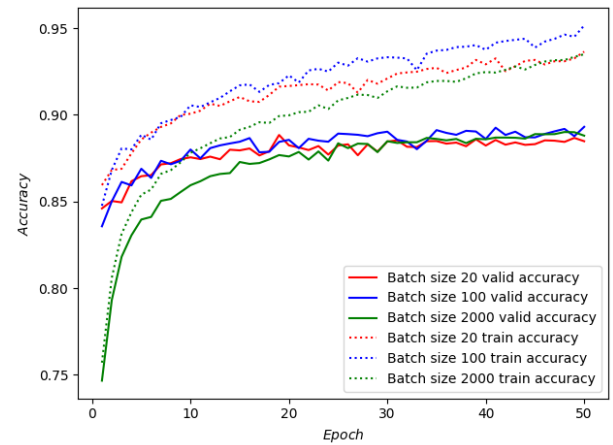
(a) Pérdida para los distintos tamaños de capas



(b) Precisión para los distintos tamaños de capas

Figura 5. **Número de neuronas por capa.** Caso 1: $n_1 = 64, n_2 = 32$; Caso 2: $n_1 = 128, n_2 = 64$; Caso 3: $n_1 = 512, n_2 = 256$;

(a) Pérdida para los distintos tamaños de batch



(b) Precisión para los distintos tamaños de batch

Figura 6. **Tamaño del batch.** Caso 1: $batch\ size = 20$; Caso 2: $batch\ size = 100$; Caso 3: $batch\ size = 2000$;